

Práctica de Búsqueda Local

Curso de Inteligencia Artificial, FIB
Herman Daniel Berrio, Marc Pérez, Artur Leivar
Marzo de 2026

ÍNDICE

1. PARTE DESCRIPTIVA.....	3
1.1. Identificación del problema.....	3
1.1.1. Descripción y planteamiento del problema.....	3
1.1.2. Análisis de las características (Naturaleza del problema).....	3
1.1.3. Descripción de los elementos del estado del problema.....	4
1.1.4. ¿Por qué es un problema de Búsqueda Local?.....	5
1.2. Estado del problema y representación.....	6
1.2.1. Justificación y descripción de la representación del problema.....	6
1.2.2. Análisis del tamaño del espacio de búsqueda.....	9
1.3. Representación y análisis de los operadores.....	10
1.3.1. Explicación de la elección de los operadores.....	10
A. El bloqueo por restricciones y la necesidad de dos operadores.....	10
B. El Optimizador Local Embebido (Micro-TSP) y el suavizado del vecindario...	10
C. Sinergia de los operadores.....	11
1.3.2. Descripción detallada de los operadores.....	11
1.3.3. Análisis del factor de ramificación.....	12
1.4. Análisis de la función heurística.....	13
1.4.1. Factores que intervienen en la heurística.....	13
1.4.2. Funciones heurísticas escogidas.....	14
1.4.3. Efectos de las funciones heurísticas en la búsqueda.....	14
1.4.4. Justificación de las ponderaciones (Weights) en la Heurística 2.....	15
1.5. Elección y generación del estado inicial.....	16
1.5.1. Descripción de los algoritmos de generación.....	16
1.5.2. Explicación y justificación de las elecciones (Coste vs. Bondad).....	16
2. PARTE EXPERIMENTAL.....	18
Experimento 1. Determinar el mejor conjunto de operadores.....	18
Experimento 2. Determinar la mejor estrategia de generación de la solución inicial.....	21
Experimento 3. Determinar los mejores parámetros para Simulated Annealing.....	24
Experimento 4. Estudiar la evolución del tiempo de ejecución siguiendo la proporción 5:100 para centros y grupos.....	27
Experimento 5. Estudiar la evolución del tiempo de ejecución de un escalado independiente de grupos y centros.....	32
Experimento 6. Estudiar la evolución del tiempo de ejecución para valores crecientes del número de helicópteros.....	36
Experimento 7. Estudiar la función heurística.....	39
3. TRABAJO DE INNOVACIÓN.....	42

1. PARTE DESCRIPTIVA

1.1. Identificación del problema

1.1.1. Descripción y planteamiento del problema

El problema nos sitúa en el escenario de una catástrofe natural donde el tiempo es un factor crítico: llegar antes o después puede suponer la diferencia entre la vida y la muerte. Se presenta un espacio físico con distintos grupos de personas repartidas aleatoriamente, de diferente tamaño y nivel de prioridad (gravedad), que han quedado aislados y necesitan ser evacuados en helicóptero.

Nuestro objetivo es gestionar la logística de una flota de helicópteros distribuidos en varios centros de rescate. Aunque encontrar una solución es aparentemente sencillo (por ejemplo, asignar helicópteros de forma aleatoria hasta recoger a todo el mundo), la calidad de la solución es lo realmente importante y encontrar la mejor solución, o saber que se ha encontrado, requiere una capacidad de cómputo que no tenemos actualmente. La distribución de los viajes debe garantizar la evacuación total respetando dos importantes restricciones por cada viaje: un límite de capacidad (máximo 15 personas por vuelo) y un límite operativo (máximo 3 grupos por vuelo antes de volver a descargar y repostar).

El objetivo final de esta asignación es minimizar el tiempo global de la operación, considerando tanto la eficiencia logística general como el tiempo máximo de llegada de los afectados más críticos (Prioridad 1).

1.1.2. Análisis de las características (Naturaleza del problema)

Este escenario presenta las siguientes características:

- Entorno observable estático: Conocemos todos los datos desde el inicio (coordenadas, distancias, prioridades), las emergencias no cambian de sitio mientras el algoritmo planifica, y el resultado de mover un helicóptero del punto A al B es 100% predecible y calculable.
- Relación con modelos clásicos: El problema base es una variante del clásico Problema del Viajante de Comercio (TSP), donde los helicópteros son los viajeros y los grupos las ciudades. Al escalar, se convierte en un mTSP (múltiples viajeros) y, finalmente, dado que hay varios centros base, estamos ante un MDVRP (Multi-Depot Vehicle Routing Problem).
- Micro-TSP interno: Adicionalmente, debido a la restricción de visitar hasta 3 grupos por viaje, cada salida del helicóptero encierra un mini-problema de permutación (un sub-TSP de 4 nodos) para decidir el orden óptimo de recogida que minimice el tiempo (y en este caso la distancia recorrida) de ese vuelo concreto.
- Objetivo: A diferencia de un TSP clásico, aquí no optimizamos distancia pura, sino tiempo absoluto. Los helicópteros sufren penalizaciones temporales (tiempo de

carga de pasajeros según prioridad y tiempo de descarga en los centros). Por tanto, una solución puede recorrer más distancia kilométrica pero ser más rápida en tiempo y por lo tanto ser una mejor solución.

- Restricciones:
 - Velocidad helicóptero 100 km/h
 - Los grupos que han de ser rescatados están compuestos por entre una y doce personas
 - Los helicópteros pueden transportar hasta 15 personas en un viaje
 - Se tarda 1 minuto en recoger a cada persona (2 minutos por persona si el grupo es de prioridad 1)
 - El cálculo de la distancia recorrida entre dos puntos se realiza mediante la distancia euclídea entre sus coordenadas.
 - Siempre se debe rescatar el grupo completo
 - Al retornar al centro de rescate, el helicóptero tarda 10 minutos en poder volver a hacer un rescate.
 - En una salida un helicóptero no podrá rescatar a más de 3 grupos.
 - Una solución debe rescatar a todos los grupos.

1.1.3. Descripción de los elementos del estado del problema

Para que el algoritmo (AIMA) pueda operar, definimos un estado como una asignación completa y válida de todos los grupos a los helicópteros. Los elementos lógicos que componen y dan significado a este estado son:

Conjunto de grupos G: coordenadas (x,y) + número de personas + prioridad

C centros de rescate: las coordenadas y el número de helicópteros por centro

Los helicópteros H: Identificados por su centro base de origen, son las entidades que se reparten la carga de trabajo.

Las Rutas: El orden cronológico en el que cada helicóptero hará sus vuelos, es decir, recogerá a los grupos asignados, en diferentes salidas.

Los Retornos a Base: Los momentos en los que el helicóptero interrumpe su ruta para volver al centro a descargar, y repostar, separando los viajes que hace (sub-rutas).

Condición de validez: Ninguna de las sub-rutas delimitadas por los retornos a base puede superar las 15 personas ni los 3 grupos.

1.1.4. ¿Por qué es un problema de Búsqueda Local?

La elección de algoritmos de mejora iterativa (Búsqueda Local) frente a algoritmos de búsqueda constructiva (como A*) se fundamenta en la propia naturaleza del estado objetivo y en las dimensiones físicas del escenario. Esta decisión se justifica por tres factores determinantes:

- **La indefinición a priori del Estado Final:** En los problemas de búsqueda clásica, el estado meta es una condición conocida desde el principio (por ejemplo, encontrar la salida de un laberinto). Sin embargo, en nuestro caso, el estado final ideal es el *óptimo global* de la función de coste, un valor que desconocemos por completo antes de ejecutar el programa. Por esta razón, es imposible programar una condición de parada tradicional (implementado en nuestro código mediante la clase `GoalTestFalse()`, que siempre retorna falso). En Búsqueda Local, la meta se redefine dinámicamente: el estado final será simplemente aquel óptimo (local o global) donde la función heurística ya no encuentre vecinos que mejoren la calidad de la solución, o cuando el algoritmo agote su tiempo de exploración.
- **Irrelevancia del camino (Path irrelevance):** En la gestión de una catástrofe, no nos interesa en absoluto la secuencia de transformaciones o pasos intermedios que el algoritmo exploró en su memoria para llegar a la planificación definitiva. El único dato valioso es el estado final en sí mismo. La Búsqueda Local nos permite operar manteniendo en memoria únicamente el estado actual y sus vecinos inmediatos, descartando el historial y ahorrando una inmensa cantidad de memoria RAM frente a los árboles de búsqueda exhaustiva.
- **Complejidad combinatoria (NP-Hard):** Como hemos comentado en el análisis del espacio de búsqueda, calcular todas las posibles combinaciones en un MDVRP de este tamaño llevaría un tiempo computacional inasumible. La Búsqueda Local nos permite sortear esta limitación navegando por este inmenso espacio de estados mediante la aplicación de operadores de vecindad (intercambios y movimientos). De este modo, el algoritmo "desciende" rápidamente por la superficie de costes hasta alcanzar una solución altamente optimizada en un tiempo de ejecución viable.

1.2. Estado del problema y representación

1.2.1. Justificación y descripción de la representación del problema

El dilema Espacio-Tiempo en Búsqueda Local

Desde el inicio, identificamos que un potencial problema de un algoritmo de Búsqueda Local es la gestión de la memoria. Durante su ejecución, algoritmos como el Hill Climbing o el Simulated Annealing instancian y evalúan una inmensa cantidad de estados sucesivos. Cualquier pequeña ineficiencia en la representación de un estado se multiplica por el factor de ramificación.

En nuestro caso, al usar Java, tenemos que tener otra cosa en cuenta, la velocidad con la que el Garbage Collector puede liberar los estados descartados. Si priorizamos únicamente la eficiencia temporal (velocidad de cálculo) añadiendo variables o estructuras de datos para ahorrar tiempo, la huella de memoria (memory footprint) podría crecer hasta saturar el Heap, ralentizando el programa por las constantes pausas del recolector de basura. Por desgracia, espacio y tiempo son criterios excluyentes. Nuestro objetivo ha sido buscar un equilibrio: una estructura lo suficientemente ligera para la memoria, pero lo bastante eficiente para que se resuelva el problema en un tiempo razonable.

Investigando un poco vemos que a diferencia de lenguajes como C, donde una matriz puede almacenarse de forma contigua en memoria y permitir accesos directos en tiempo constante, en Java los arrays de arrays no garantizan contigüidad. Cada nivel introduce una referencia adicional. Esto implica realizar múltiples desreferenciaciones para acceder a los datos, lo que puede empeorar la localidad de memoria y, en consecuencia, el rendimiento, por lo que es ligeramente mejor tener dos arrays independientes que una matriz, si no hace falta hacer cálculos extra para localizar un elemento, es decir, el id del elemento (un grupo o helicóptero por ejemplo), coincide con su posición en el array. Esta particularidad del lenguaje la tendremos en cuenta para los datos estáticos y dinámicos.

Datos Estáticos vs. Dinámicos

La primera decisión para optimizar este equilibrio fue dividir la información, ver qué datos son exclusivos de una solución, y cuales no cambian y se pueden compartir.

La mayor parte de los datos los hemos declarado como **estáticos** y compartidos para todas las instancias de los sucesores:

- **Información base:** Generada por la biblioteca desastres o introducida por el usuario y que no cambia entre soluciones:
 - Información de centros y grupos, número de helicópteros por centro, la semilla de aleatoriedad, todas las opciones del programa...
- **Información que deriva de la anterior:** Esta información que se calcula a partir de información estática del problema la necesitan todos los estados sucesores y no

cambia, si la calculamos al principio ganaremos mejoramos eficiencia temporal y espacial.

- El tiempo de recogida de cada grupo, matrices de distancias y tiempos de viaje entre grupos y/o centros.

De este modo, la memoria que realmente debemos vigilar y controlar es la que modifica cada suceso y esta se reduce exclusivamente a la **información dinámica**: el reparto exacto de los grupos en los helicópteros.

Evolución de la Representación Dinámica

Para representar esta información dinámica (el estado *per se*), pasamos por varias iteraciones buscando el máximo rendimiento:

- **Fase 1: Estructura jerárquica (Descartada)**
Nuestra primera idea fue utilizar estructuras jerárquicas: un array de centros → un array de helicópteros → un array de viajes → estructuras de grupos.

Solo con plantearlo ya vemos que no es una buena solución, pero nos sirvió como punto de partida.
- **Fase 2: Aplanamiento (Representación Final)**
Nos dimos cuenta de que todos los centros tienen el mismo número de helicópteros. Esto nos permite prescindir del concepto "Centro" en el estado dinámico y tener una lista plana de helicópteros. Con esto no solo simplificamos la estructura sino que nos ahorramos desreferenciaciones. Además, separar los viajes en sub-arrays añadía no solo objetos que luego tendrían que ser eliminados añadiendo presión sobre el Garbage Collector, especialmente en nuestro caso, con la generación de sucesores. Y por último también reducimos complejidad en los operadores

La Representación Final Adoptada:

Decidimos aplanar todo en una matriz primitiva bidimensional: `public int[][] rutas;`

- Las filas representan los helicópteros.
- Las columnas contienen los identificadores enteros ($ID \geq 0$) de los grupos a rescatar.

Dado que el identificador de un grupo es siempre cero o positivo, utilizamos el valor negativo -1 como delimitador lógico para marcar el retorno obligatorio al centro base a descargar. Así, una secuencia plana como [5, 12, -1, 8, -1] separa y define los distintos viajes sin necesidad de anidar estructuras de datos complejas.

Esta implementación tiene en cuenta el orden de los grupos dentro de un viaje, ya que siempre es estrictamente relevante (afecta a la distancia de vuelo euclidiana), pero el orden cronológico de los distintos viajes de un helicóptero sólo es relevante para la Segunda Heurística, donde el momento temporal exacto en el que se rescata al último herido de

Prioridad 1 altera el coste final, esta implementación también nos permite tener ese orden en cuenta.

Por último, esta estructura primitiva nos permite una clonación muy eficiente de la memoria. Al generar sucesores, utilizamos la función nativa de Java `System.arraycopy`, logrando copias exactas del estado con un coste computacional ínfimo en comparación con la instanciación clásica de objetos.

Alternativas descartadas en favor del equilibrio

Durante el desarrollo, valoramos dos optimizaciones que fueron descartadas:

- **Caché local de tiempos:** Antes era obvio que pre calcular las distancias o tiempos entre puntos era una mejora de coste computacional muy grande frente al precio que se pagaba en espacio (cuando el problema adquiere un cierto espacio de búsqueda), pues solo había una copia static y nos ahorramos calcularlo en cada sucesor, ahora estamos ante una situación similar, donde consideramos guardar dentro del estado un vector con el tiempo total ya calculado de cada helicóptero y una variable que sea la suma de estos. Aunque ahorraría tiempo al aplicar operadores (recalculando solo los helicópteros modificados), replicar este vector y su variable de suma total en todos los estados sucesores, suponer un incremento en el uso de memoria que no es estrictamente necesario, pues es información que se puede calcular, es una opción que valoramos implementar si el programa fuese lento y no mostrará problemas de memoria, pero al probar el programa vemos que sin esta optimización de cálculo, el programa es suficientemente rápido.
- **Codificación a nivel de Bits (Bitwise):** Pensando en posibles mejoras y sin más ideas para mejorar nuestra implementación, consultamos con la IA (gemini), y nos da una representación para optimizar al máximo el espacio pero con un coste computacional muy alto, tener un array de tamaño G (número de grupos), cada elemento del array podría ser un short (dependiendo del tamaño del número de helicópteros y el número de grupos, se podría cambiar el tipo de dato), y codificar la información en bits, Ejemplo con short:
 - Estructura del Estado: short[] estadoGrupos (tamaño G).
 - ¿Cómo funciona?: El índice del array es el ID del grupo. El valor dentro del array codifica quién y cuándo lo rescata. Usando operaciones a nivel de bits (Bitwise), un short (16 bits) puede guardar:
 - Bits 0-5: ID del Helicóptero (hasta 64 helicópteros).
 - Bits 6-11: ID del Viaje para ese helicóptero (hasta 64 viajes).
 - Bits 12-15: Posición en el viaje (1º, 2º o 3º).

A pesar de que con esta representación un estado ocupa literalmente unos pocos bytes y podemos tener cientos de miles de estados en memoria sin problema, recorrer esto para calcular la suma de tiempos de los helicópteros sería muy lento.

1.2.2. Análisis del tamaño del espacio de búsqueda

En un problema de Búsqueda Local, el espacio de búsqueda se define como el conjunto completo de todas las configuraciones posibles (estados) que el algoritmo puede visitar durante su ejecución al aplicar los operadores definidos. En nuestro escenario, este espacio no consiste simplemente en asignar grupos a helicópteros, sino que está formado por todas las permutaciones y asignaciones posibles de los G grupos de afectados a la flota de H helicópteros, incluyendo el orden secuencial exacto en el que son rescatados y los momentos precisos en los que el vehículo retorna a la base para descargar.

Cálculo del tamaño

El diseño de nuestra representación ligera (basada en matrices de primitivos) se justifica plenamente al dimensionar la magnitud de este espacio. Si ignoramos temporalmente las restricciones de capacidad, el problema equivale matemáticamente a distribuir G elementos distintos en H secuencias ordenadas. El número de configuraciones resultantes viene dado por la fórmula de variaciones con repetición y particiones:

$$Espacio \approx \frac{(G+H-1)!}{(H-1)!}$$

Para una instancia estándar del experimento con G=100 grupos y H=5 helicópteros, el cálculo resulta en:

$$\frac{104!}{4!} \approx 4.29 \times 10^{165} \text{ estados posibles}$$

A esta cifra astronómica hay que sumarle la complejidad posicional de los delimitadores de viaje (-1). Dado que la restricción operativa impone un máximo de 3 grupos por vuelo, es matemáticamente obligatorio intercalar, como mínimo absoluto, 34 retornos a base (100/3) a lo largo de las secuencias, lo que multiplica aún más las combinaciones topológicas de los arrays.

Conclusión del Espacio

Aunque es cierto que las duras restricciones logísticas del problema (máximo 15 personas por vuelo y máximo 3 grupos) actúan como un filtro natural que invalida una inmensa mayoría de estas combinaciones teóricas, el subconjunto de estados válidos sigue superando cómodamente los 10^{100} estados.

Este volumen inabarcable ratifica nuestra decisión arquitectónica: utilizar matrices primitivas veloces y algoritmos heurísticos probabilísticos es la única vía computacionalmente viable para navegar de forma eficiente por un espacio que, de otro modo, resultaría incomputable.

1.3. Representación y análisis de los operadores

1.3.1. Explicación de la elección de los operadores

En Búsqueda Local, un operador no solo es una transformación matemática que genera un estado válido a partir de otro, sino que es la herramienta que define la **topología del vecindario** por el que se moverá el algoritmo. La elección del "tamaño de paso" de estos operadores es crítica: dos operadores teóricamente capaces de alcanzar cualquier solución pueden hacer que algoritmos de escalada estricta, como el *Hill Climbing*, se comporten de forma radicalmente distinta y converjan en óptimos locales diferentes.

Partiendo de nuestra representación basada en matrices, diseñamos la estrategia de vecindad abordando tres grandes retos:

A. El bloqueo por restricciones y la necesidad de dos operadores

Nuestra primera aproximación fue utilizar un único operador abstracto de movimiento (**MOVE**), capaz de extraer un grupo y reinsertarlo en cualquier otro punto del tablero (creando, reordenando o eliminando viajes). Teóricamente, si moviéramos un grupo cada vez sin filtrar por heurística, podríamos recorrer todo el espacio de soluciones.

Sin embargo, en la práctica, las estrictas restricciones del problema (máximo 3 grupos por viaje) generan "muros" infranqueables en el paisaje de búsqueda. Supongamos un estado donde dos helicópteros alejados tienen viajes llenos (3 grupos) pero geográficamente cruzados. Un operador MOVE jamás podría intercambiar un grupo del Helicóptero A al Helicóptero B, porque al intentar el primer movimiento violaría la restricción de 4 grupos máximos, y el estado sería descartado. Para hacer este intercambio con MOVE, el algoritmo tendría que crear un viaje "temporal" vacío para hacer hueco, un paso intermedio que empeora drásticamente la heurística y que el *Hill Climbing* jamás aceptaría.

Para solucionar este estancamiento (*deadlock* lógico), surge la necesidad estricta de implementar el operador **SWAP**, capaz de permutar dos grupos simultáneamente, esquivando las restricciones de capacidad sin generar estados inválidos intermedios.

B. El Optimizador Local Embebido (Micro-TSP) y el suavizado del vecindario

Analizando la topología de los viajes, observamos que cada ruta individual constituye un sub-problema del Viajante de Comercio (TSP). Dado que un viaje tiene un máximo de 3 grupos, estamos ante un TSP de 4 nodos (Centro + 3 Grupos). Al usar distancias euclidianas fijas, el espacio de combinaciones sin contar simetrías es de tan solo 3 permutaciones posibles. Por tanto, decidimos aplicar un optimizador de fuerza bruta en tiempo constante directamente en la generación del estado: cada vez que se modifica un viaje de 3 grupos, el código evalúa matemáticamente cuál de las 3 ordenaciones es la más corta y la aplica automáticamente.

Impacto topológico del optimizador: En un primer momento, consideramos esta optimización como opcional. Sin embargo, al activarla, observamos un fenómeno contraintuitivo: el *Hill Climbing* arrojaba resultados peores que cuando estaba desactivada.

Al forzar que cada viaje estuviese siempre ordenado de forma óptima desde su concepción, alteramos la definición fundamental de lo que constituye una "solución" en nuestro espacio. La implementación de este optimizador, provocó que el Hill Climbing convergiera prematuramente a peores óptimos locales debido al suavizado del vecindario.

La "rugosidad" del espacio original (permitir viajes desordenados) abría caminos secundarios que el HC usaba para transitar hacia mejores óptimos globales. Al eliminar estos estados intermedios subóptimos, el algoritmo se quedaba atascado más rápido. Pese a que esta decisión perjudica al HC (y beneficia teóricamente al *Simulated Annealing*), el enunciado exige explícitamente que los helicópteros recojan a los grupos "*siguiendo el recorrido más corto*", por lo que esta optimización no es una ventaja estratégica, sino una restricción del dominio del problema que debe aplicarse obligatoriamente a todo estado válido.

C. Sinergia de los operadores

La elección conjunta de **SWAP + MOVE** responde al principio fundamental de la Búsqueda Local: la garantía de que, partiendo de cualquier estado válido inicial, los operadores nos permiten alcanzar cualquier otro estado válido del espacio, incluido el óptimo global. Hemos seleccionado esta dupla por su extrema sinergia:

1. **El operador SWAP es un optimizador de ajuste fino:** Es excelente para intercambiar personas críticas (Prioridad 1) por grupos de baja prioridad, equilibrando el peso cronológico o la distancia sin desestructurar el número de viajes del helicóptero. Sin embargo, sin el apoyo de otro operador, la IA jamás podría vaciar un helicóptero ineficiente.
2. **El operador MOVE es un mutador estructural:** Permite trasladar carga bruta de un helicóptero saturado a uno ocioso. Es el único capaz de reducir la cantidad de viajes totales o construir nuevos viajes si las restricciones de capacidad lo exigen.

Ambos operadores se complementan perfectamente: MOVE cambia la topología "gruesa" del plan de rescate, mientras que SWAP reordena y afina la eficiencia dentro de esa topología.

1.3.2. Descripción detallada de los operadores

Los dos operadores estructurales: **SWAP** (Intercambio) y **MOVE** (Movimiento). Ambos operan directamente sobre la matriz bidimensional del estado y delegan la corrección de los viajes a una función de limpieza (*limpiarYReestructurar()*).

Operador 1: SWAP (Intercambio de grupos)

- **Descripción:** Selecciona dos grupos distintos y permuta sus posiciones en la matriz. El intercambio puede darse entre dos grupos del mismo helicóptero (optimizando una ruta local) o entre dos helicópteros distintos (balanceando la carga global).
- **Condiciones de aplicabilidad:** Tras la permutación y la reestructuración automática de los -1 y el orden de los grupos en los viajes, el estado resultante debe

superar el test de validez (esValido()), es decir, ningún viaje afectado por el cambio puede exceder las 15 personas ni los 3 grupos.

- **Efectos:** Su ejecución es espacialmente neutra (no modifica la longitud total de los arrays de los helicópteros implicados, solo los valores de sus índices).

Operador 2: MOVE (Extracción e Inserción)

- **Descripción:** Extrae un grupo (G a) de su posición actual en el helicóptero de origen H1 y lo inserta en una nueva posición arbitraria, dentro del mismo helicóptero o de uno distinto H2.
- **Condiciones de aplicabilidad:** La posición de destino no puede ser idéntica a la de origen. El estado resultante debe ser válido tras reestructurar los viajes.
- **Efectos:** Este operador realiza una transformación estructural profunda. Destruye el array de origen y construye un array de destino, mediante copias ultrarrápidas de memoria (System.arraycopy). Su efecto principal es que puede alterar el número de viajes de un helicóptero (creando viajes nuevos o vaciando viajes ineficientes).

1.3.3. Análisis del factor de ramificación

El factor de ramificación (b) define cuántos estados vecinos (sucesores) se generan a partir de un estado padre. Su cálculo es estrictamente dependiente del operador y del tamaño del problema. Sea G el número total de grupos (ej. 100) y H el número de helicópteros totales (ej. 5):

- **Ramificación del operador SWAP:**
El número máximo teórico de intercambios posibles es la combinación de G elementos tomados de 2 en 2.

$$b_{swap} = \binom{G}{2} = \frac{G \times (G - 1)}{2}$$

Para 100 grupos, el operador SWAP evalúa aproximadamente 4.950 vecinos.

- **Ramificación del operador MOVE:**
Se selecciona 1 grupo entre G posibles. Ese grupo se extrae, reduciendo el tamaño total de los grupos distribuidos en rutas a G-1. El número de huecos de inserción posibles a lo largo de todos los helicópteros es la suma de los grupos restantes más los puntos iniciales/finales de cada helicóptero (aproximadamente G + H).

$$b_{move} \approx G \times (G + H)$$

Para 100 grupos y 5 helicópteros, el operador MOVE evalúa aproximadamente a 10.500 vecinos.

Conclusión del factor: En el algoritmo *Hill Climbing*, donde la exploración es exhaustiva, el factor de ramificación total (el vecindario (SWAP + MOVE)) asciende a aproximadamente a 15,450 sucesores por iteración.

Si contrastamos este vecindario local con el tamaño del espacio de búsqueda total (que superaba los 10^{165} estados), vemos el verdadero poder de nuestra aproximación: estamos evaluando una fracción infinitesimal del espacio total. Al haber programado los operadores utilizando primitivos y `System.arraycopy`, el algoritmo es capaz de evaluar y comparar estos 15.000 vecinos en cuestión de milisegundos, descendiendo por la topología de costes en un tiempo computacionalmente viable.

Por el contrario, en el Simulated Annealing, el factor de ramificación explícito para el motor de búsqueda se reduce matemáticamente a $b = 1$, ya que el algoritmo simplemente selecciona aleatoriamente una de esas 15.450 combinaciones posibles y decide si transita hacia ella sin necesidad de generar ni evaluar el resto del vecindario.

1.4. Análisis de la función heurística

1.4.1. Factores que intervienen en la heurística

El objetivo de nuestras funciones heurísticas es traducir una configuración de un estado o solución (asignaciones de helicópteros y rutas) a un **único** valor numérico (coste) que el algoritmo intentará minimizar, ya que AIMA sólo acepta un valor como heurístico.

Para que esta traducción sea precisa, la heurística computa los siguientes factores físicos y logísticos del problema para cada viaje:

- **Factor de Distancia/Velocidad:** Las distancias euclidianas entre centros y grupos se traducen a tiempo de vuelo asumiendo una velocidad constante de 100 km/h.
- **Factor de Carga (Prioridad):** El tiempo que el helicóptero pasa posado en tierra recogiendo al grupo. Este factor penaliza a los grupos de alta prioridad (Prioridad 1), ya que requieren el doble de tiempo de carga (2 minutos por persona frente a 1 minuto para los no prioritarios).
- **Factor de Descarga (Retorno a base):** Cada vez que se lee un delimitador de fin de viaje (-1), se suma el tiempo de vuelo de regreso al centro de origen más un factor constante de 10 minutos si luego vuelve a hacer otro viaje (tiempo de descargar y repostar).
- **Orden de recogida de un vuelo:** Aunque el enunciado nos dice que el orden de recogida de un vuelo debe ser el más corto para considerarse solución, diferentes combinaciones de grupos cambian el tiempo de vuelo total.
- **Orden de los vuelos:** A pesar de que el enunciado nos dice que no consideramos los vuelos en paralelo, estos si que nos afectarían a la segunda heurística, donde priorizaríamos tener en los primeros viajes de los helicópteros a los grupos de mayor prioridad, ya que cuanto más tiempo tardemos más nos penalizará en el coste.

La suma de estos factores a lo largo de los arrays nos da el tiempo exacto de operación de cada helicóptero. A partir de este cálculo base, diseñamos dos funciones heurísticas distintas.

1.4.2. Funciones heurísticas escogidas

Atendiendo a los requisitos, hemos implementado dos heurísticas diferentes:

Heurística 1: Minimización del coste total

- **Justificación:** Representa el enfoque clásico de los problemas VRP (*Vehicle Routing Problem*). Consiste en sumar el tiempo total de operación de todos los helicópteros juntos. El objetivo en el mundo real sería minimizar el tiempo de rescate, pero en problemas sin heridos podría ser, minimizar el consumo de combustible global, el desgaste de la flota y las horas totales de conducción de los trabajadores.

Heurística 2: Optimización Multiobjetivo (Coste total + urgencia grupos de prioridad 1)

- **Justificación:** En una emergencia real, no solo nos podemos centrar en una medida concreta, podemos tener una solución donde tardemos más en recoger a todos los grupos pero asegurándonos que los heridos lleguen antes al hospital y dándoles más posibilidades de recuperarse. Esta heurística mantiene la suma de tiempos totales, pero le añade una penalización basada en el **tiempo máximo de llegada del último grupo de Prioridad 1**. Justificamos esta elección porque obliga a la Inteligencia Artificial a no tratar a todos los nodos por igual, introduciendo una variable de moralidad, en un algoritmo que, por naturaleza, es insensible.

1.4.3. Efectos de las funciones heurísticas en la búsqueda

El cambio de una heurística a otra transforma por completo la topología por el que se mueven el *Hill Climbing* y el *Simulated Annealing*, generando planes de vuelo muy distintos:

- **Efectos de la Heurística 1 (H1):** Al usar H1, el algoritmo moldea las rutas buscando la máxima densidad geográfica. Los helicópteros llenarán sus cabinas al máximo (cerca de 15 personas) agrupando nodos que estén físicamente muy juntos, sin importar si son de prioridad alta o baja. El efecto visual es el de rutas circulares muy eficientes. Sin embargo, puede provocar que un grupo de Prioridad 1 sea rescatado en el último viaje del día simplemente porque estaba geográficamente alejado del centro, lo cual es logísticamente eficiente pero operativamente inaceptable en una emergencia.
- **Efectos de la Heurística 2 (H2):** H2 distorsiona el paisaje de búsqueda creando fuertes "pendientes" (penalizaciones) cuando un nodo de Prioridad 1 se encuentra al

final de un array de ruta. El efecto principal en el algoritmo es que los operadores de *Swap* y *Move* que trasladen grupos de Prioridad 1 hacia las primeras posiciones de los viajes (y a los primeros viajes del día) serán aceptados masivamente. El resultado es que los helicópteros a menudo harán primeros viajes menos eficientes en capacidad (ej. saliendo a rescatar solo a 4 personas urgentes y volviendo), sacrificando eficiencia geométrica global a cambio de un rescate crítico temprano.

1.4.4. Justificación de las ponderaciones (Weights) en la Heurística 2

Para implementar la Heurística 2, nos enfrentamos a un problema matemático: los algoritmos de Búsqueda Local solo pueden minimizar un único número. Para optimizar dos objetivos simultáneos, recurrimos a una combinación lineal ponderada:

$$H(s) = T_{total} + (w * T_{max P1})$$

Dónde T_{total} es la suma de todos los vuelos, $T_{max P1}$ es el tiempo en el que llega el último herido grave, y w es el peso de ponderación.

Justificación del uso de ponderaciones:

Si no usáramos un peso multiplicador ($w = 1.0$), el objetivo de la urgencia sería ignorado por la IA. Esto ocurre porque el T_{total} es una suma masiva de todos los helicópteros (un número muy grande), mientras que $T_{max P1}$ es solo un valor temporal de un único helicóptero (un número comparativamente pequeño).

Justificación del valor del peso:

El peso w actúa como una "tasa de cambio" entre eficiencia y urgencia. Al establecer un peso de 5.0, le estamos diciendo matemáticamente al algoritmo: *"Retrasar 1 minuto la llegada de un herido grave al hospital es igual de perjudicial que gastar 5 minutos de vuelo"*.

- Si usáramos un peso demasiado bajo ($w = 0.1$), el algoritmo seguiría comportándose como la Heurística 1, primando el tiempo total.
- Si usáramos un peso demasiado alto ($w = 1000$), el algoritmo sufriría de hiperfocalización: destrozando las rutas por completo, enviando helicópteros vacíos a la otra punta del mapa solo para ganar unos segundos con un Prioridad 1, resultando en una mala solución

Un valor intermedio nos garantiza rutas globalmente eficientes donde, en caso de empate logístico, los grupos de Prioridad 1 siempre son empujados hacia el inicio de la jornada operativa, aunque el valor de la ponderación no debería ser algo fijo, se debería modificar en función de la gravedad de los heridos o de cada situación concreta, hacerlo una variable nos da flexibilidad.

1.5. Elección y generación del estado inicial

Para que los algoritmos de Búsqueda Local (*Hill Climbing* y *Simulated Annealing*) puedan comenzar a explorar el espacio de estados, necesitan un punto de partida válido (una solución). En nuestro diseño (clase *Board*), hemos implementado dos algoritmos generadores distintos que el usuario puede seleccionar por parámetros: un generador **Aleatorio** y un generador **Greedy (Voraz)**.

1.5.1. Descripción de los algoritmos de generación

Ambos algoritmos comparten un motor base para garantizar que el estado inicial devuelto sea siempre 100% válido según las reglas del problema (máximo 15 personas y máximo 3 grupos por viaje). Iteran sobre el array de grupos y, al asignar un grupo a un helicóptero, verifican si se superan los límites. Si es así, insertan automáticamente un delimitador de retorno a base (-1) y resetean los contadores lógicos de capacidad antes de añadir al nuevo grupo.

La diferencia entre ambos radica en **cómo eligen a qué helicóptero va cada grupo**:

- **Algoritmo Aleatorio (*generarSolucionInicialAleatoria*):** Carece por completo de inteligencia espacial. Para cada uno de los grupos a rescatar, genera un número entero aleatorio entre 0 y el número total de helicópteros de la flota. El grupo se asigna a ese helicóptero independientemente de en qué centro geográfico se encuentre aparcado.
- **Algoritmo Greedy (*generarSolucionInicialGreedy*):** Aplica una heurística espacial constructiva. Para cada grupo, recorre la matriz precalculada estática de distancias comparando el grupo actual con todos los centros disponibles. Una vez encuentra el centro geográficamente más cercano, asigna el grupo al primer helicóptero disponible que pertenece a dicho centro.

1.5.2. Explicación y justificación de las elecciones (Coste vs. Bondad)

La implementación de estas dos estrategias no es redundante, sino que responde a las diferentes necesidades teóricas de los algoritmos de búsqueda que vamos a utilizar. Analizamos cada una en base a su coste computacional y a la calidad heurística de la solución resultante:

Análisis del Estado Inicial Aleatorio:

- **Coste de hallar la solución:** Extremadamente bajo. Su complejidad temporal es lineal $O(G)$, donde G es el número de grupos. Se limita a iterar una vez el array invocando a la clase Random.
- **Bondad de la solución:** Pésima (coste heurístico altísimo). Al no considerar la geografía, genera rutas caóticas donde los helicópteros cruzan el mapa de punta a punta constantemente para recoger grupos alejados, solapando sus trayectorias.

- **Justificación de uso:** A pesar de su mala calidad inicial, esta generación es vital para el Simulated Annealing (SA). El SA es un algoritmo probabilístico que necesita *diversidad* para explorar ampliamente el espacio de búsqueda. Si siempre empezara desde un estado lógicamente ordenado, limitaría su exploración a un área concreta del paisaje topológico. Empezar con rutas aleatorias le garantiza arrancar desde puntos radicalmente distintos en cada ejecución, evitando caer sistemáticamente en el mismo óptimo local.

Análisis del Estado Inicial Greedy:

- **Coste de hallar la solución:** Ligeramente superior al aleatorio, pero marginal en tiempos absolutos. Su complejidad es $O(G * C)$, ya que por cada uno de los G grupos, debe realizar C comparaciones (número de centros) en la matriz de distancias. Al ser operaciones de lectura sobre un array primitivo estático, no supone un coste de cómputo relevante.
- **Bondad de la solución:** Muy buena geográficamente, pero logísticamente desbalanceada. Cada centro sólo atiende a su zona de influencia, lo que reduce drásticamente el tiempo de vuelo global. Sin embargo, al asignar los rescates iterativamente al *primer* helicóptero de cada centro, satura a este vehículo dejándolo con decenas de viajes mientras el resto de la flota del centro permanece en tierra.
- **Justificación de uso:** Esta generación está diseñada específicamente para el Hill Climbing (HC). Como el HC es un algoritmo intensivo (evalúa miles de sucesores por paso) y de escalada estricta (solo acepta movimientos que mejoren la heurística), darle un estado aleatorio le haría perder una inmensa cantidad de iteraciones corrigiendo cruces de rutas obvios. Al darle el estado Greedy, el HC parte de un escenario geográficamente sensato y puede dedicar todos sus esfuerzos computacionales (mediante el operador MOVE) a balancear la carga, repartiendo los viajes del helicóptero saturado hacia los helicópteros vacíos del mismo centro.

Cabe destacar que, aunque teóricamente un estado inicial aleatorio permitiría explorar óptimos locales distintos (estrategia utilizada en variantes como el Random-Restart Hill Climbing), en el contexto de una ejecución simple, el HC desperdiciaría la inmensa mayoría de sus iteraciones resolviendo ineficiencias geográficas triviales (cruces de rutas). Arrancar desde un estado Greedy sitúa al algoritmo directamente en una cuenca de atracción de alta calidad, permitiéndole focalizar sus recursos computacionales en el balanceo logístico fino.

2. PARTE EXPERIMENTAL

Experimento 1. Determinar el mejor conjunto de operadores

a. Hipótesis:

El mejor conjunto de operadores es **"swap+move"**. Anticipamos que la combinación de ambos permitirá una exploración más completa del espacio de estados, evitando que el algoritmo quede atrapado prematuramente en óptimos locales que un solo operador no podría superar.

b. Condiciones y metodología:

Para realizar el siguiente experimento, se ha utilizado el algoritmo de **Hill Climbing** sobre un escenario de 100 grupos y 5 centros de rescate (con 1 helicóptero por centro). Se han realizado **10 repeticiones (réplicas)** para cada configuración, utilizando semillas aleatorias para garantizar que la comparación entre los operadores (**swap**, **move** y **swap+move**) se realice sobre los mismos datos de entrada en cada réplica. El criterio de optimización ha sido la **Heurística 1**.

c. Resultados:

Los resultados obtenidos muestran que la combinación de operadores influye directamente en la calidad de la solución y en el coste temporal:

Operadores	Coste medio (min)	Tiempo medio (ms)
swap	~2688.52	~538
move	~2585.28	~1268
swap + move	~2526.46	~1943

Boxplot de Coste: Demuestra que la combinación **swap+move** es la más eficaz para alcanzar los costes más bajos y estables, superando las limitaciones de los operadores individuales.

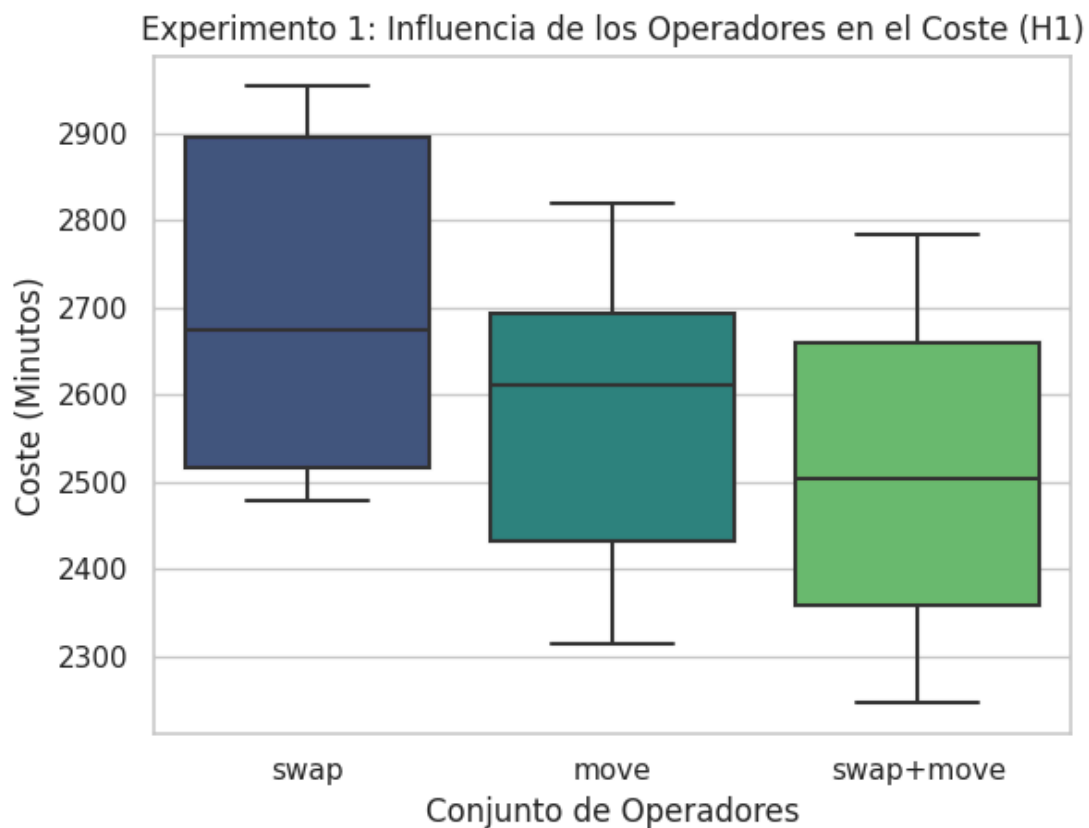
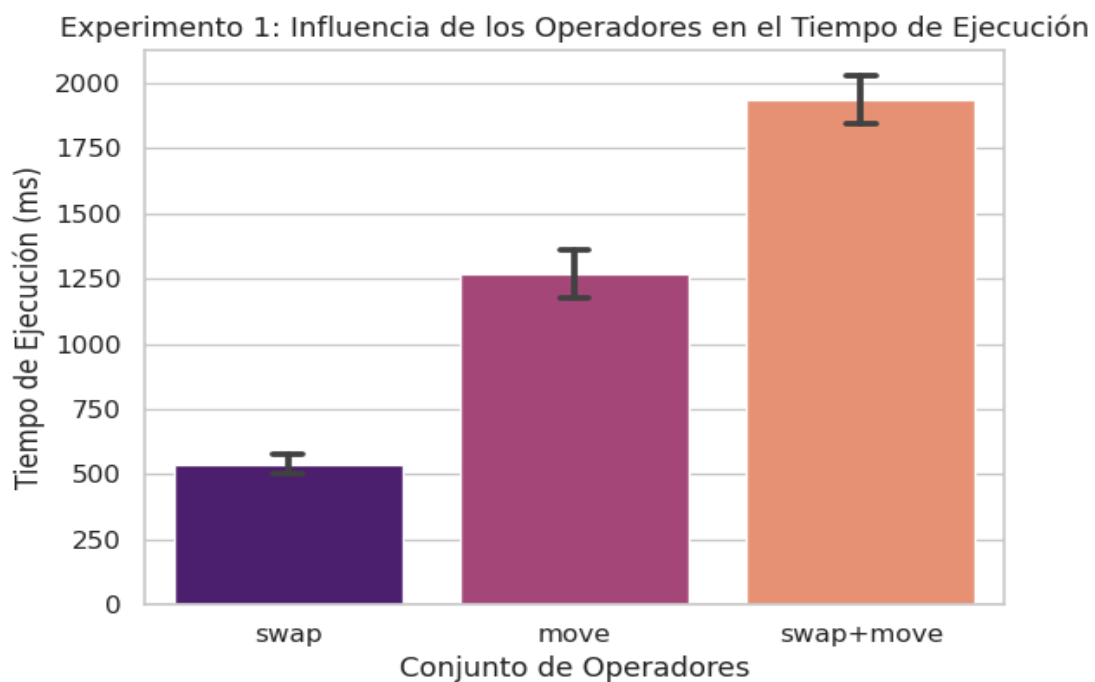


Gráfico de Barras de Tiempo: Refleja que el uso de ambos operadores incrementa el tiempo de ejecución debido al aumento del factor de ramificación y del número de sucesores a evaluar.



d. Conclusiones:

Nosotros esperábamos que los mejores operadores fuesen **"swap+move"**, ya que teóricamente ofrecen una mayor conectividad en el espacio de búsqueda al permitir tanto el intercambio de grupos como la reorganización de la cantidad de grupos por viaje.

Los resultados obtenidos lo confirman: el conjunto **"swap+move"** ha logrado el coste más bajo en todas las ejecuciones. Hemos observado que, aunque el tiempo de ejecución aumenta debido a un mayor factor de ramificación (el algoritmo debe evaluar más sucesores en cada paso), la mejora en la calidad de la solución justifica este coste computacional.

Por lo tanto, fijamos esta configuración de operadores para el resto de los experimentos de la práctica.

Experimento 2. Determinar la mejor estrategia de generación de la solución inicial

a. Hipótesis:

El mejor método de inicialización para este problema es el "**Greedy**". Nuestra hipótesis es que, al generar un punto de partida con una estructura lógica (asignando grupos cercanos y respetando prioridades desde el inicio), el algoritmo de búsqueda local convergerá más rápido y hacia un óptimo local de mayor calidad (menor coste) que partiendo de una solución aleatoria.

b. Condiciones y metodología:

En este experimento se han comparado las dos estrategias de generación del estado inicial implementadas: **aleatorio** y **greedy**.

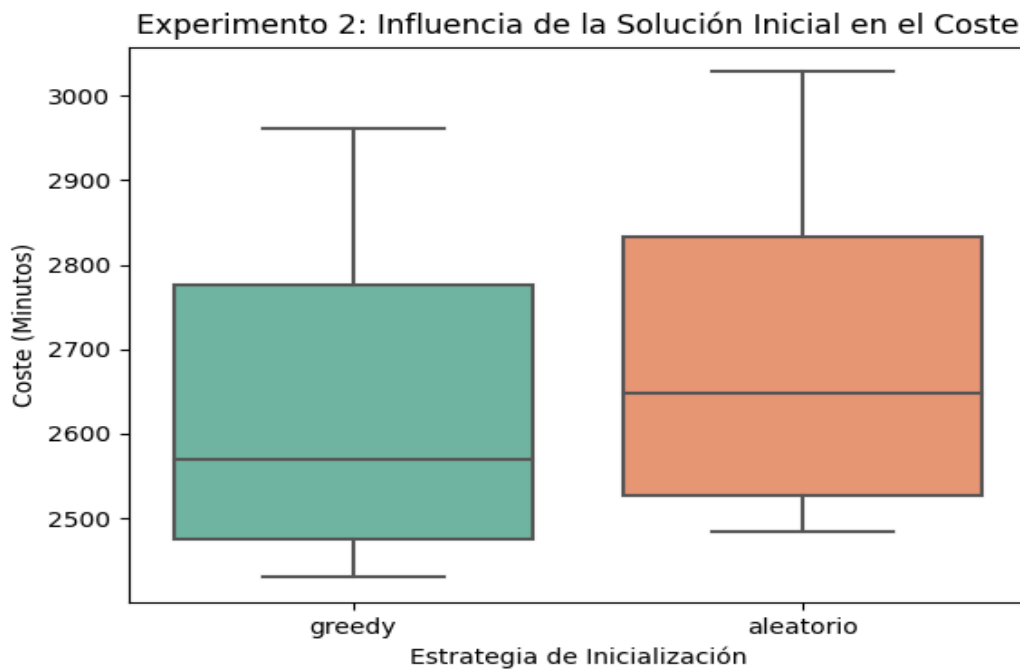
- **Escenario:** 100 grupos, 5 centros de rescate y 1 helicóptero por centro.
- **Configuración fija:** Basándonos en el Experimento 1, hemos fijado los operadores a **swap+move**.
- **Algoritmo:** Hill Climbing con la Heurística 1.
- **Muestra:** Se han realizado 10 réplicas con las mismas semillas aleatorias para ambos métodos, garantizando una comparación justa.

c. Resultados:

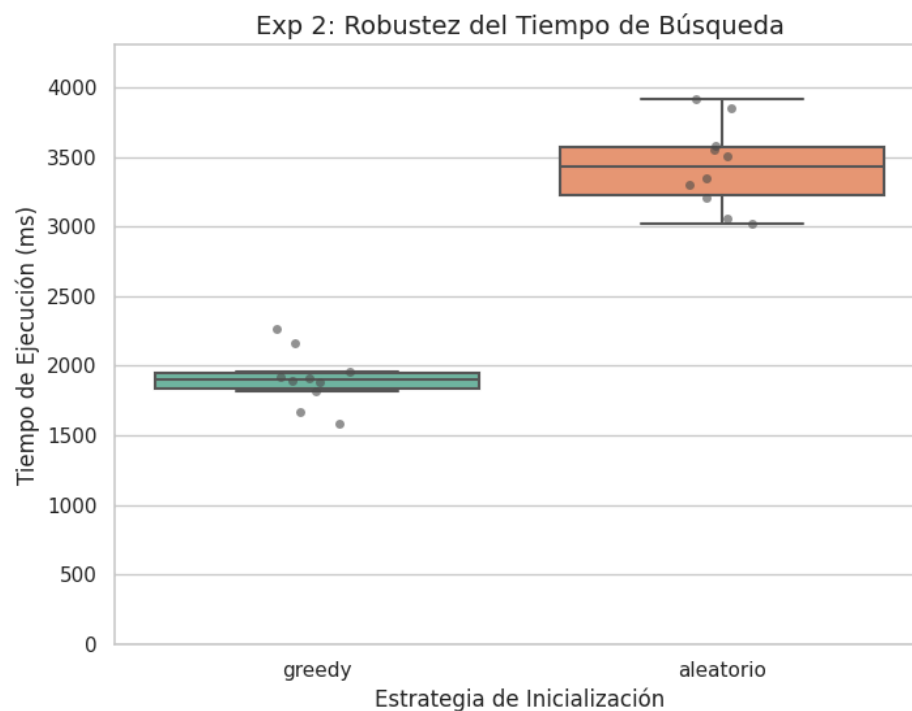
Los datos recolectados muestran una ventaja clara para la aproximación greedy. Observando que la solución inicial Greedy no solo empieza con un coste menor, sino que reduce casi a la mitad el tiempo de búsqueda (1.9s vs 3.4s) necesario para alcanzar el óptimo local.

Inicialización	Coste medio (min)	Desv. Estándar (Coste)	Tiempo medio (ms)
Greedy	2629.27	182.44	1907.0
Aleatorio	2693.75	187.35	3434.7

Boxplot de Coste: El boxplot evidencia que la inicialización **greedy** es una estrategia mucho más robusta y fiable que la aleatoria para situar al algoritmo cerca de un buen óptimo.



Boxplot de Tiempo: Confirma la eficiencia a partir de una solución greedy, la cual reduce drásticamente el camino de búsqueda.



d. Conclusiones:

Nosotros esperábamos que la inicialización **"Greedy"** fuese mejor debido a que sitúa al algoritmo en una situación cercana al objetivo global .

Los resultados obtenidos confirman plenamente esta expectativa. La diferencia de coste (~64 minutos de ahorro en media) y, sobre todo, la eficiencia temporal, validan el uso de heurísticas constructivas para generar el estado inicial. La inicialización aleatoria obliga al Hill Climbing a realizar un recorrido mucho más largo y costoso por el espacio de estados, aumentando la probabilidad de quedar atrapado en óptimos locales mediocres.

Así que, fijamos la inicialización a **greedy** para todos los experimentos restantes.

Experimento 3. Determinar los mejores parámetros para Simulated Annealing

a. Hipótesis:

Esperamos que el **Simulated Annealing** supere al Hill Climbing al permitir la aceptación de estados peores con una probabilidad controlada. Anticipamos que un número elevado de pasos y un enfriamiento lento (**lambda** pequeño) permitirán una exploración exhaustiva del espacio de búsqueda antes de converger.

b. Condiciones y metodología:

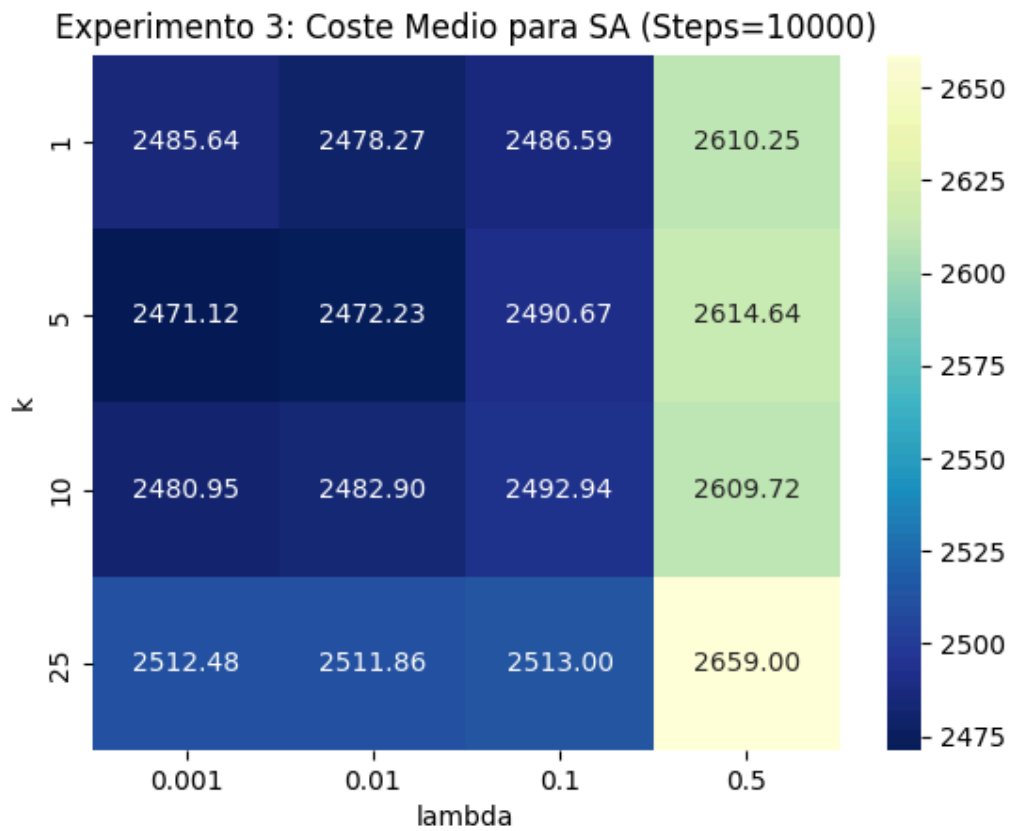
Se han mantenido fijos los operadores (**swap+move**) y la inicialización (**greedy**). Se ha realizado un barrido de los siguientes parámetros: **steps** (1000, 5000, 10000), **k** (1, 5, 10, 25) y **lambda** (0.001, 0.01, 0.1, 0.5). Cada combinación se ha ejecutado 10 veces con las mismas semillas que en los experimentos anteriores para asegurar la comparabilidad.

c. Resultados:

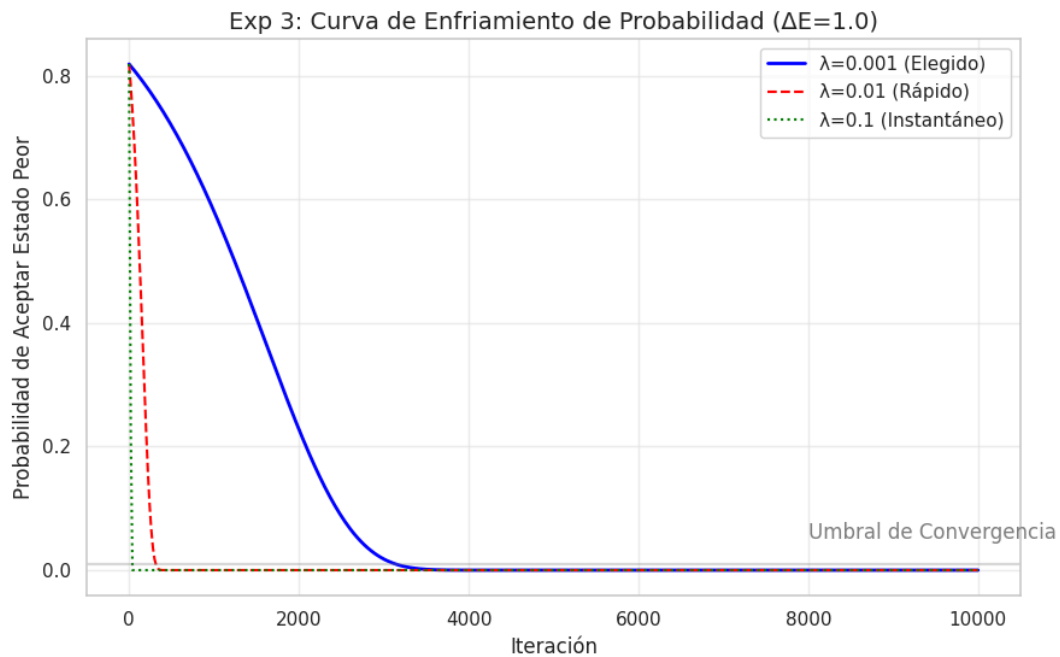
Los mejores resultados se concentran en el bloque de 10.000 iteraciones:

Steps	K	Lambda (λ)	Coste Medio (min)	Tiempo Medio (ms)
10000	1	0.01	2478.27	286.5
10000	5	0.001	2471.12	289.7
10000	10	0.001	2480.95	297.0

Mapa de Calor: Este mapa de calor nos ayuda a identificar visualmente la región de parámetros (en concreto **k=5** y **lambda=0.001**) donde el algoritmo logra escapar de óptimos locales y obtener resultados superiores al Hill Climbing.



Curva de Enfriamiento: Ilustra la transición teórica del algoritmo desde una fase de exploración amplia hacia una de exploración intensiva y menos 'permisiva' a medida que el número de iteraciones aumenta.



d. Conclusiones:

Nosotros esperábamos que el SA encontrara mejores soluciones que el HC, y los resultados lo confirman: el coste se reduce un **6% aproximadamente**. Observamos que con **lambda=0.5** el algoritmo se "enfria" demasiado rápido (el tiempo de ejecución cae a ~80ms porque deja de aceptar cambios pronto), comportándose casi como un Hill Climbing y obteniendo peores resultados (~2600+). Los valores de **lambda=0.001** mantienen la capacidad de exploración durante más tiempo, logrando los mejores mínimos.

Experimento 4. Estudiar la evolución del tiempo de ejecución siguiendo la proporción 5:100 para centros y grupos

a. Hipótesis:

Esperamos que, a medida que aumenta el tamaño del problema, el tiempo de ejecución de Hill Climbing aumente de manera descontrolada. En cambio, anticipamos que el tiempo de ejecución del algoritmo Simulated Annealing prácticamente sea constante, comparado con el de Hill Climbing, debido a lo acotado que está este por sus parámetros.

También esperamos que, aunque aumente el tamaño del problema, el coste de la solución encontrada de ambos algoritmos sea parecido.

b. Condiciones y metodología:

Se han mantenido fijos los operadores (**swap+move**) y la inicialización (**greedy**). Para Simulated Annealing, se han mantenido los mejores parámetros obtenidos en el anterior experimento (steps = 10000, k = 5, $\lambda = 0.001$).

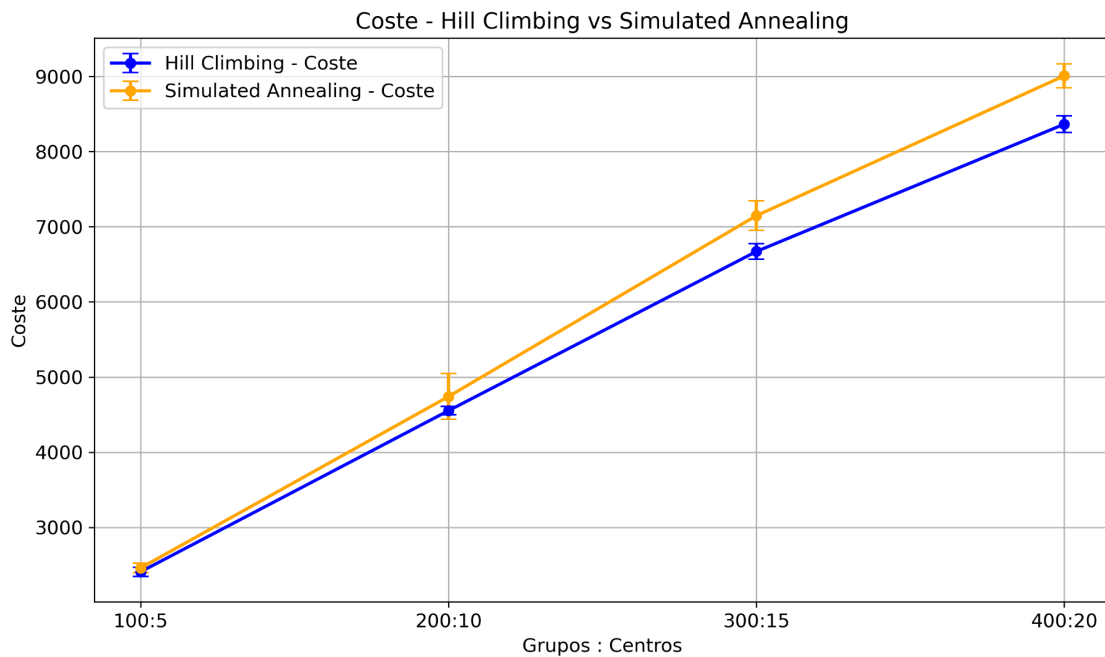
Se ha realizado el experimento para Hill Climbing y para Simulated Annealing. Para ambos se ha ido aumentando el número de grupos en +100 y el número de centros en +5 en cada iteración, siguiendo la proporción 5:100 de centros:grupos. Aún así,

- Para Hill Climbing, se han realizado 4 iteraciones, ya que a partir de ahí el tiempo de ejecución era demasiado grande.
- Para Simulated Annealing, se han realizado 12 iteraciones, para observar mejor la tendencia del tiempo de ejecución.

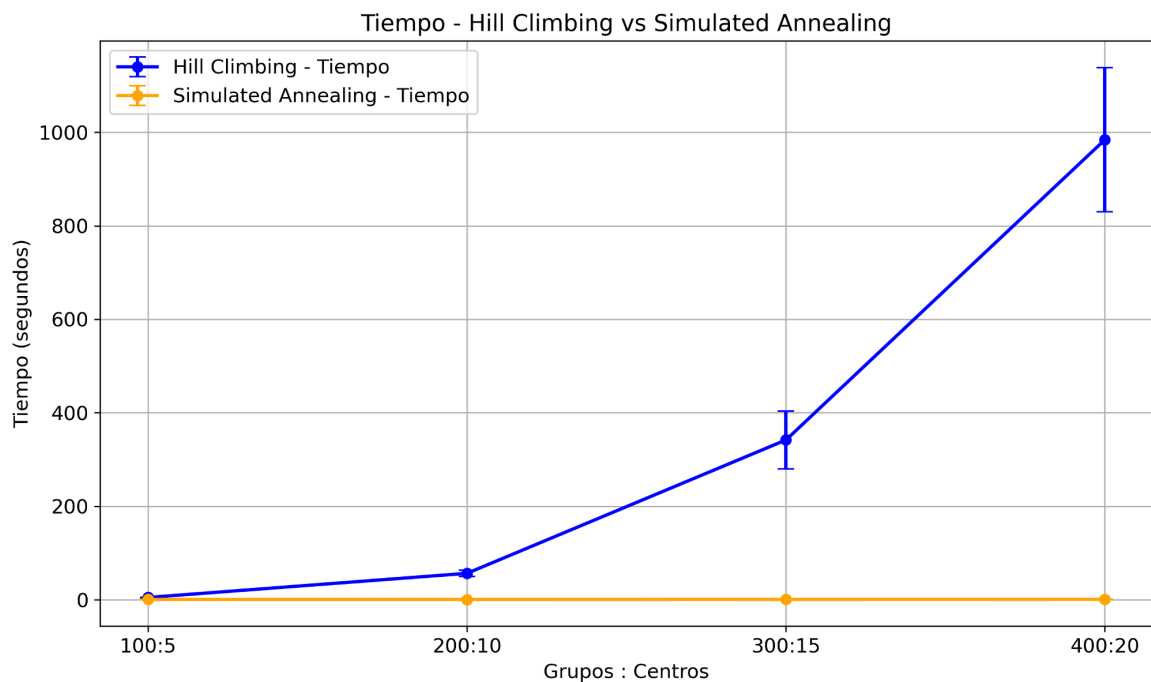
Cada combinación se ha ejecutado 10 veces para obtener resultados concluyentes del experimento.

c. Resultados:

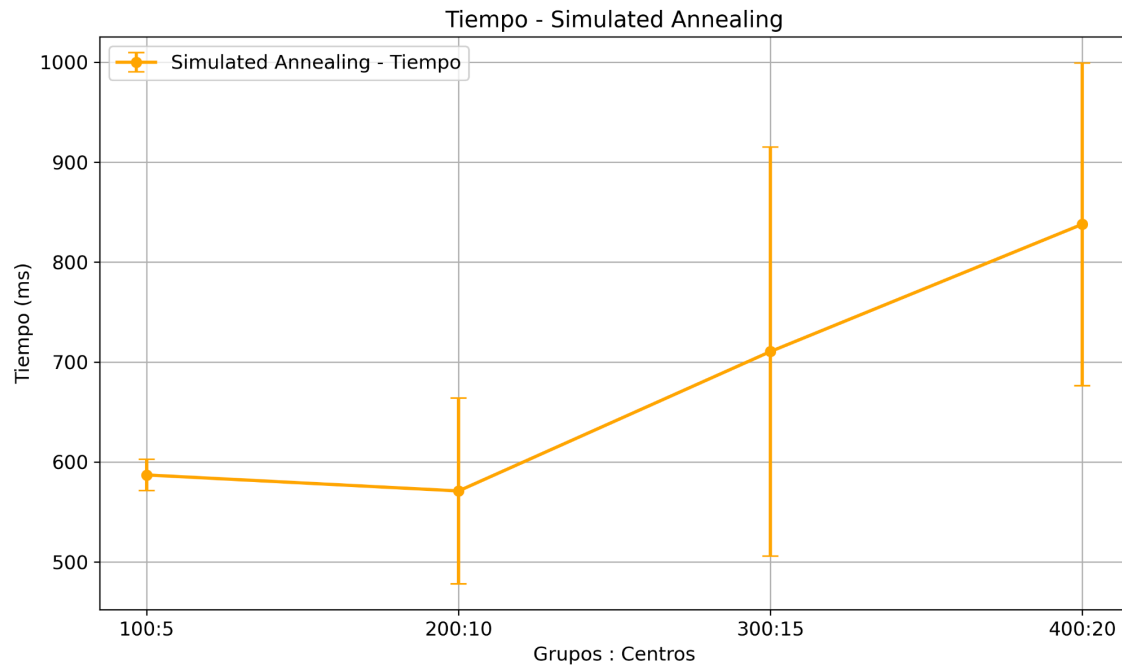
En el siguiente gráfico se puede observar la relación del **coste de la solución** con el tamaño del problema, para ejecuciones con Hill Climbing y para ejecuciones con Simulated Annealing.



En el siguiente gráfico se puede observar la relación del **tiempo de ejecución del algoritmo** con el tamaño del problema, para ejecuciones con Hill Climbing y para ejecuciones con Simulated Annealing.

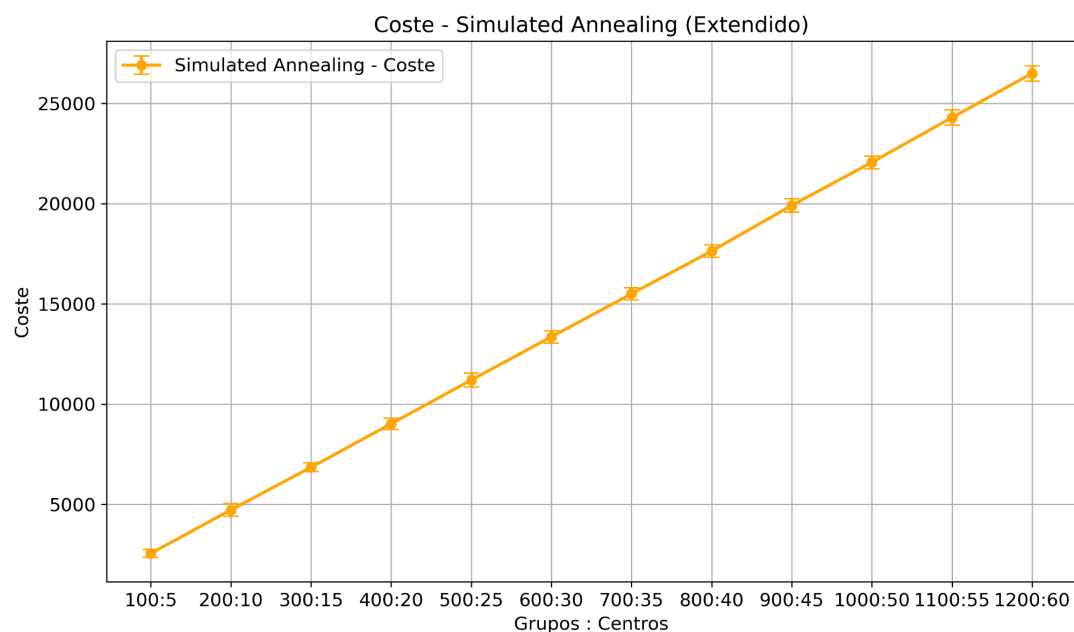


En el siguiente gráfico se puede observar la relación del **tiempo de ejecución del algoritmo** con el tamaño del problema. Es un 'zoom' del gráfico anterior, para ver con precisión los valores de **Simulated Annealing**.

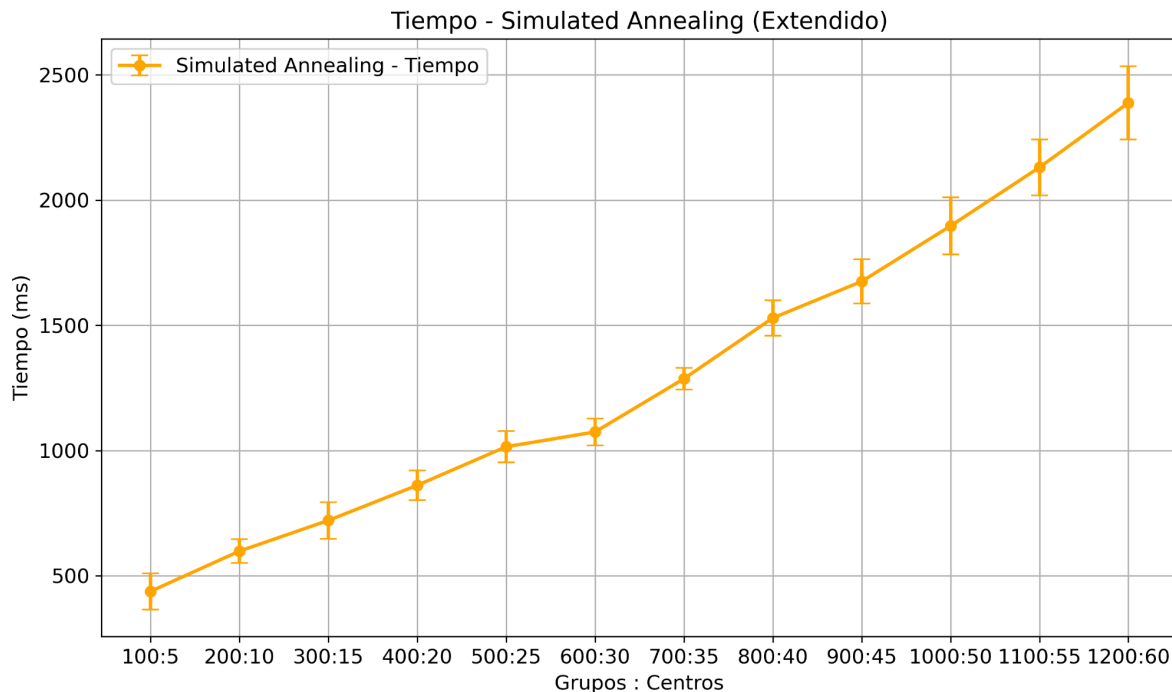


En el siguiente gráfico se puede observar la relación del **coste de la solución** con el tamaño del problema, para ejecuciones con Simulated Annealing.

En este caso, se han realizado 12 ejecuciones para observar mejor el comportamiento de Simulated Annealing.



En el siguiente gráfico se puede observar la relación del **tiempo de ejecución del algoritmo** con el tamaño del problema, para ejecuciones con Simulated Annealing. Corresponden al tiempo de las 12 iteraciones del gráfico anterior.



d. Conclusiones:

En relación al coste:

El coste de ambos, Hill Climbing y Simulated Annealing, es mayor cuanto mayor es el tamaño del problema. A partir de los valores de gráfico se puede deducir **que el coste de la mejor solución encontrada crece linealmente** a medida que aumentan los valores de grupos y centros con una misma proporción (en este caso 100:5).

También podemos concluir que el coste encontrado por **Simulated Annealing suele ser un poco peor que el encontrado por Hill Climbing**.

En relación al tiempo:

Se observa que el tiempo de ejecución de **Hill Climbing aumenta rápidamente, casi de manera exponencial**. Pasa de ejecutarse en cuestión de milisegundos para 100 grupos a tardar casi 17 minutos para 400 grupos.

Por el contrario, el tiempo de ejecución de Simulated Annealing se mantiene casi constante comparado con el de Hill Climbing. La gráfica (Tiempo - HC vs SA) muestra que, mientras que Hill Climbing tarda cientos de miles de ms, SA se mantiene todo el rato en las unidades de los cientos de ms.

Extensión, 12 iteraciones solo de Simulated Annealing

Con la ejecución de las 12 iteraciones de SA se puede analizar mejor el comportamiento de este.

Las dos conclusiones más importantes son que, en primer lugar, el coste de la mejor solución encontrada aumenta **linealmente** con respecto al tamaño del problema. Y en segundo lugar, que el tiempo de ejecución también aumenta linealmente.

Experimento 5. Estudiar la evolución del tiempo de ejecución de un escalado independiente de grupos y centros

a. Hipótesis:

Creemos que el tiempo de ejecución será extremadamente sensible al número de **grupos**, presentando un crecimiento posiblemente cuadrático. Esto se debe a que el factor de ramificación de los operadores swap y move depende directamente de las combinaciones entre grupos ($O(G^2)$). En cambio, esperamos que el aumento de **centros** no afecte significativamente al tiempo de ejecución, ya que no multiplica el número de elementos a reorganizar, aunque sí debería reducir el coste total al ofrecer bases más cercanas.

b. Condiciones y metodología:

Se ha utilizado el algoritmo **Hill Climbing** con los operadores **swap+move** y la inicialización **greedy**. El experimento se divide en dos bloques:

- **Escenario A (Grupos):** Se fijan 5 centros y se escala de 100 a 300 grupos (+50 por paso).
- **Escenario B (Centros):** Se fijan 100 grupos y se escala de 5 a 25 centros (+5 por paso).

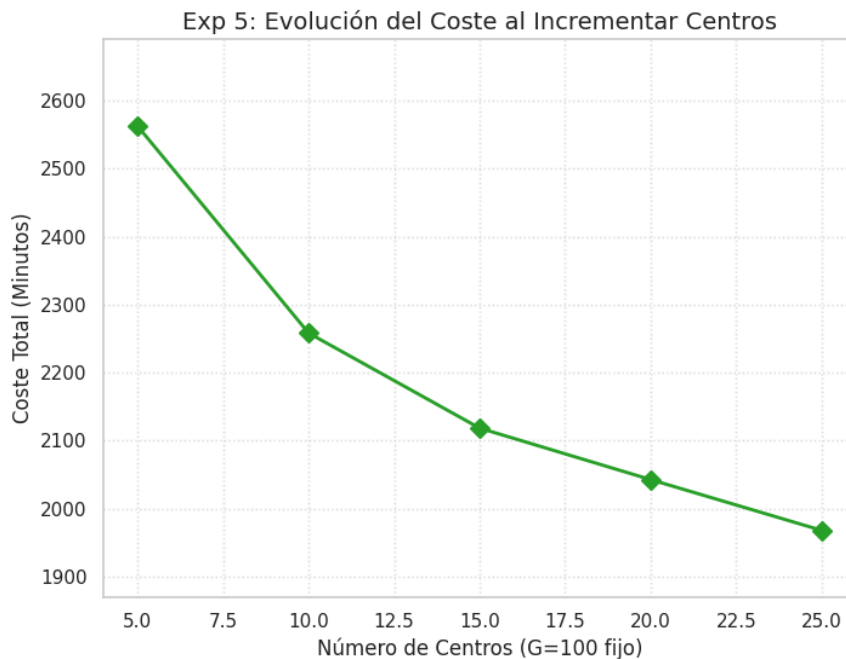
Para mitigar la varianza, se han realizado **10 réplicas** por cada punto de control utilizando las mismas semillas en ambos escenarios.

c. Resultados:

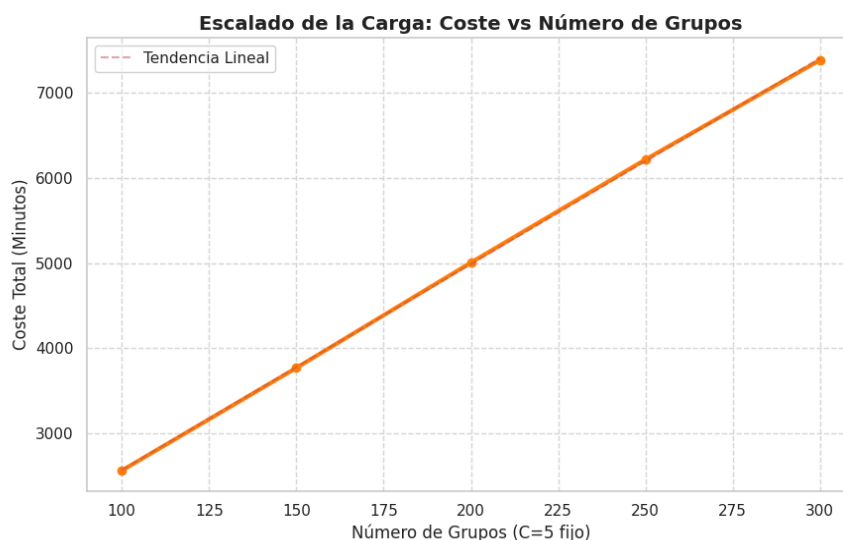
En el siguiente gráfico se analiza la comparativa de escalado. Se puede observar cómo la curva roja (Grupos) se dispara exponencialmente mientras la azul (Centros) permanece casi plana.

Variable	Valor Mín	Valor Máx	Tiempo Inicial (ms)	Tiempo Final (ms)	Incremento Tiempo
Grupos (G)	100	300	2.161	182.403	~84x
Centros (C)	5	25	2.100	2.202	~1,04x

En el siguiente gráfico, tratamos de observar cómo afecta el número de centros, manteniendo el número de grupos fijo. Observamos que la curva desciende de forma clara, lo que indica que **una mejor infraestructura (más bases) permite rutas más cortas y eficientes**.

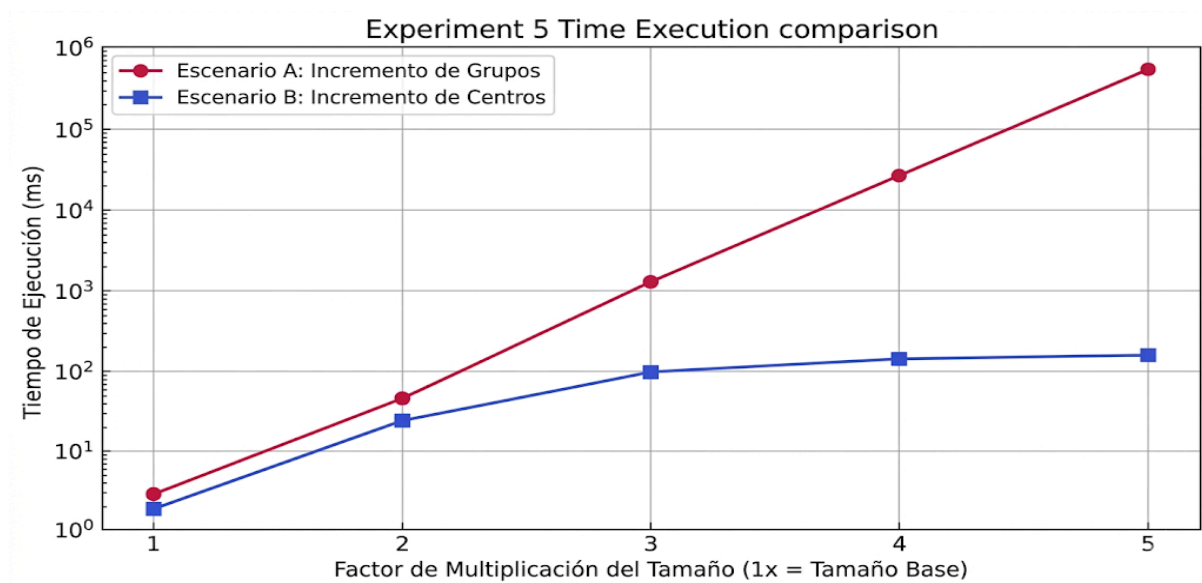


En el siguiente gráfico, se muestra la relación entre la cantidad de personas a rescatar y el coste temporal del operativo. La tendencia es **lineal y predecible**, lo que valida que **la función heurística H1 mide correctamente el volumen de trabajo**: a doble de víctimas, aproximadamente el doble de tiempo total de operación.



El siguiente gráfico compara el aumento del tiempo de ejecución de forma **relativa**, multiplicando el tamaño base (1x, que corresponde a $G = 100$ y $C = 5$) hasta cinco veces (5x, lo que sería $G = 500$ y $C = 25$).

La curva roja (Grupos) vemos cómo se dispara a medida que aumentan, demostrando que **añadir grupos multiplica la carga de trabajo computacional**. La línea azul (Centros) permanece casi plana, confirmando que el algoritmo es muy eficiente al gestionar más bases logísticas y que este factor no penaliza el rendimiento.



d. Conclusiones:

En relación al tiempo:

Los resultados confirman una asimetría total en la complejidad. Mientras que triplicar los grupos (100 -> 300) multiplica el tiempo de ejecución por **84**, quintuplicar los centros (5 -> 25) apenas varía el tiempo (manteniéndose en el rango de los 2 segundos).

Esto demuestra que el cuello de botella computacional es el **factor de ramificación**, el cual crece cuadráticamente con el número de grupos, mientras que el número de centros solo añade opciones de destino que no necesariamente penalizan el cálculo de la función sucesora.

En relación al coste:

- **Escenario A (Grupos):** El coste aumenta de forma **lineal** con respecto al número de grupos. Cada nuevo grupo añade una carga de trabajo y un tiempo de desplazamiento/recogida adicional que se suma de forma constante al total.
- **Escenario B (Centros):** El coste disminuye a medida que añadimos centros (de 2.562 a 1.967 minutos), lo que valida la eficiencia de tener bases distribuidas. Sin embargo, se observa un **rendimiento decreciente**: la mejora de pasar de 5 a 10 centros es mucho mayor que la de pasar de 20 a 25, sugiriendo que llega un punto

donde el factor limitante es la capacidad de los helicópteros y no la distancia a la base.

Experimento 6. Estudiar la evolución del tiempo de ejecución para valores crecientes del número de helicópteros

a. Hipótesis:

Aumentar el número de helicópteros mejorará el coste, ya que, entre otras cosas, el tiempo de espera entre cada viaje (10 minutos) no deberá realizarse tantas veces.

También, como incrementar el número de helicópteros aumenta el espacio de búsqueda, probablemente el tiempo de ejecución de Hill Climbing aumentará. Aunque tampoco mucho debido a nuestra representación del problema.

En último lugar, creemos que es totalmente distinto a aumentar el número de centros con un helicóptero, debido a que:

- más centros dividen mejor la cobertura geográfica, reduciendo costes y distancias.
- más helicópteros reducen tiempos de espera, pero no mejoran la cobertura geográfica.

b. Condiciones y metodología:

Se han mantenido fijos los operadores (**swap+move**) y la inicialización (**greedy**).

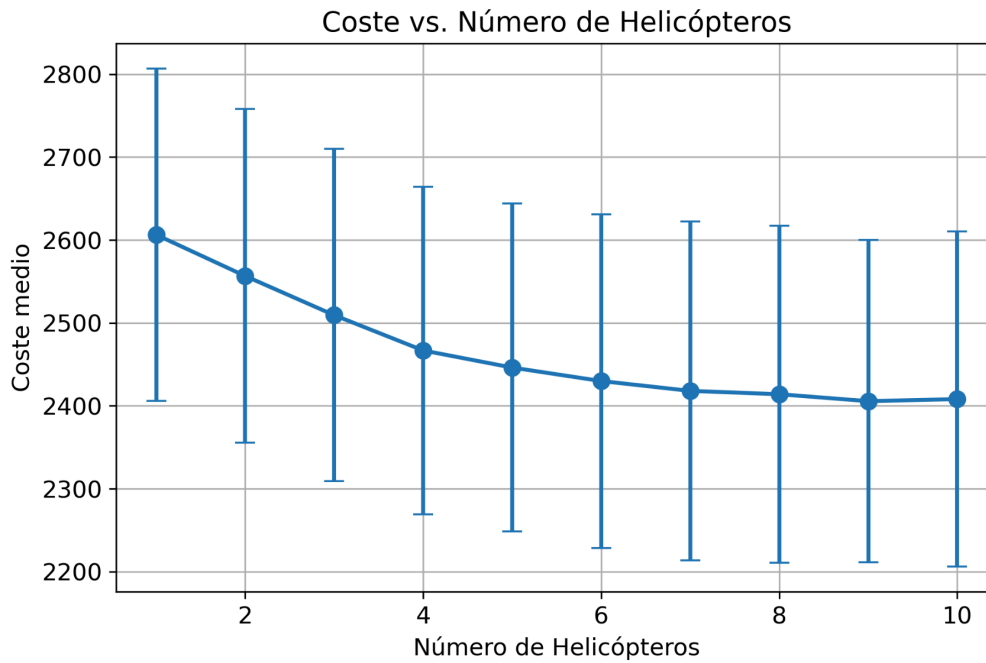
Se ha realizado el experimento para Hill Climbing. Se ha ido aumentando el número de helicópteros por centro de 1 en 1, hasta llegar a 10 helicópteros por centro.

Se medirá el coste de la solución encontrada y el tiempo de la ejecución en milisegundos.

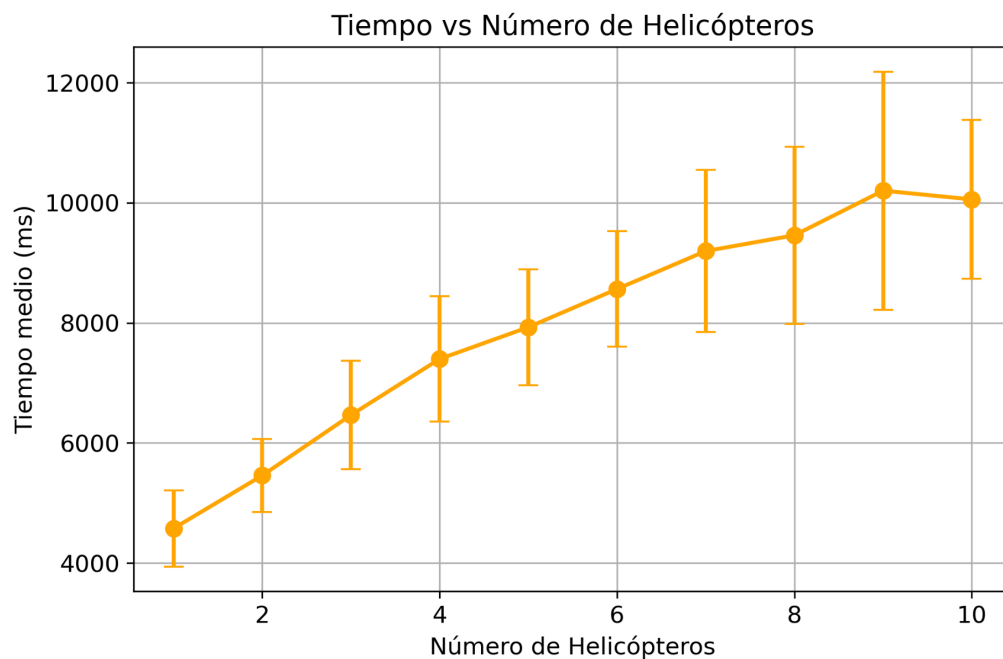
Cada combinación se ha ejecutado 10 veces para obtener resultados concluyentes del experimento.

c. Resultados:

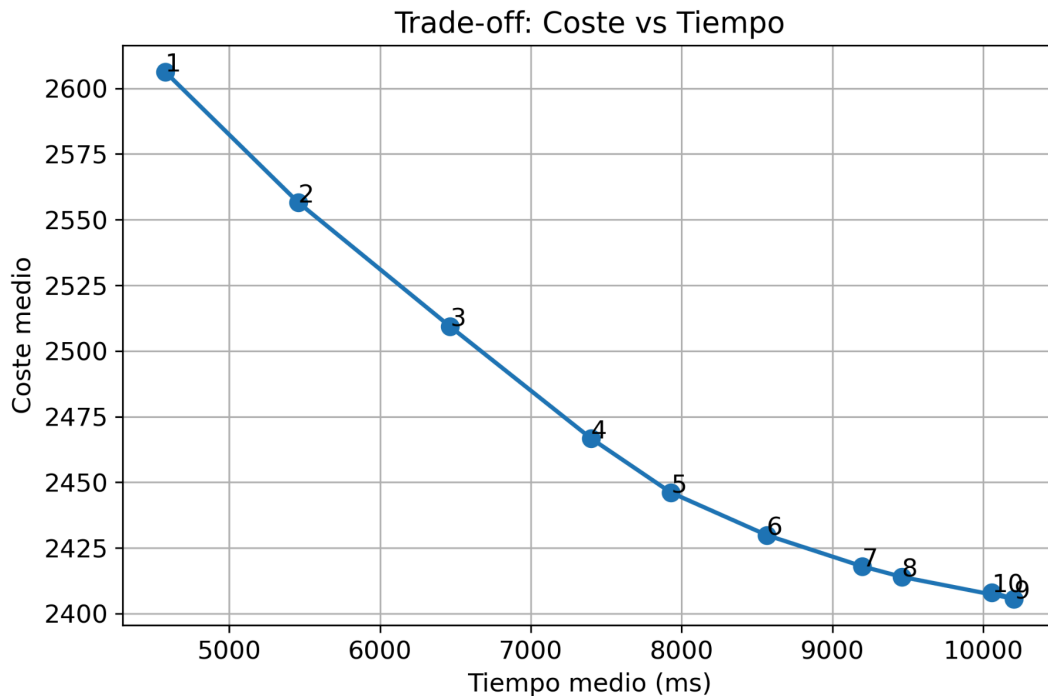
En el siguiente gráfico se puede observar la relación del **coste medio de la solución** con el número de helicópteros.



En el siguiente gráfico se puede observar la relación del **tiempo de ejecución del algoritmo** con el número de helicópteros.



En el siguiente gráfico se puede observar la relación entre coste y tiempo de ejecución a medida que se va aumentando el número de helicópteros.



d. Conclusiones:

La primera conclusión que podemos extraer de los resultados es que, como se había supuesto, incrementar el número de helicópteros disminuye el coste de la solución encontrada. Aunque quizás no tanto como se hubiera esperado en un principio, ya que la mejora se estabiliza a partir de cierto número; a partir de 7-8 helicópteros.

Esta estabilización se puede observar también, aunque con significado diferente, en el gráfico *Trade-off: Coste vs Tiempo*, donde a partir de cierto punto más tiempo de ejecución no implica mejor coste. Se observa que la curva de la función se va haciendo más plana en el 5, y posteriormente en el 7.

La segunda conclusión que podemos extraer es que **el tiempo de ejecución aumenta con el número de helicópteros, reflejando el crecimiento del espacio de búsqueda.**

Aun así, la función no es completamente lineal: hay fluctuaciones debido a la naturaleza heurística del Hill Climbing.

Experimento 7. Estudiar la función heurística

a. Hipótesis:

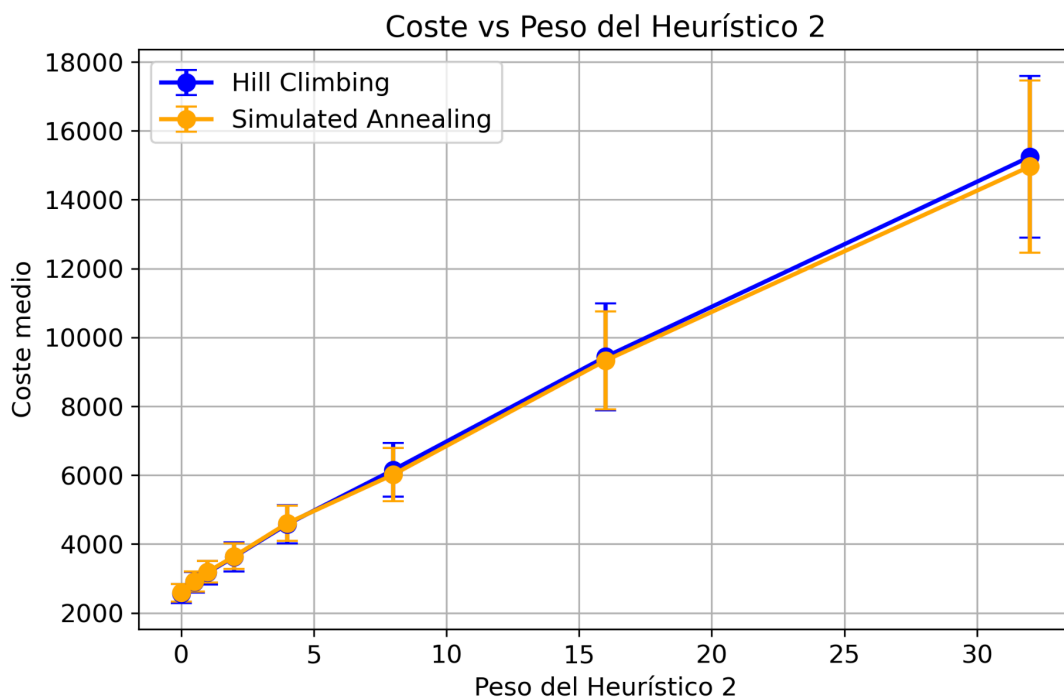
Esperamos que, al aumentar el peso w de la heurística **H2**, los algoritmos prioricen el rescate temprano de los grupos de prioridad 1 (heridos). Esto debería resultar en una disminución del tiempo máximo de llegada de estos grupos al centro de rescate, aunque previsiblemente aumentará el tiempo total de la operación (**H1**). También anticipamos que el coste heurístico final (la suma ponderada) crecerá linealmente con w debido a la magnitud de la penalización.

b. Condiciones y metodología:

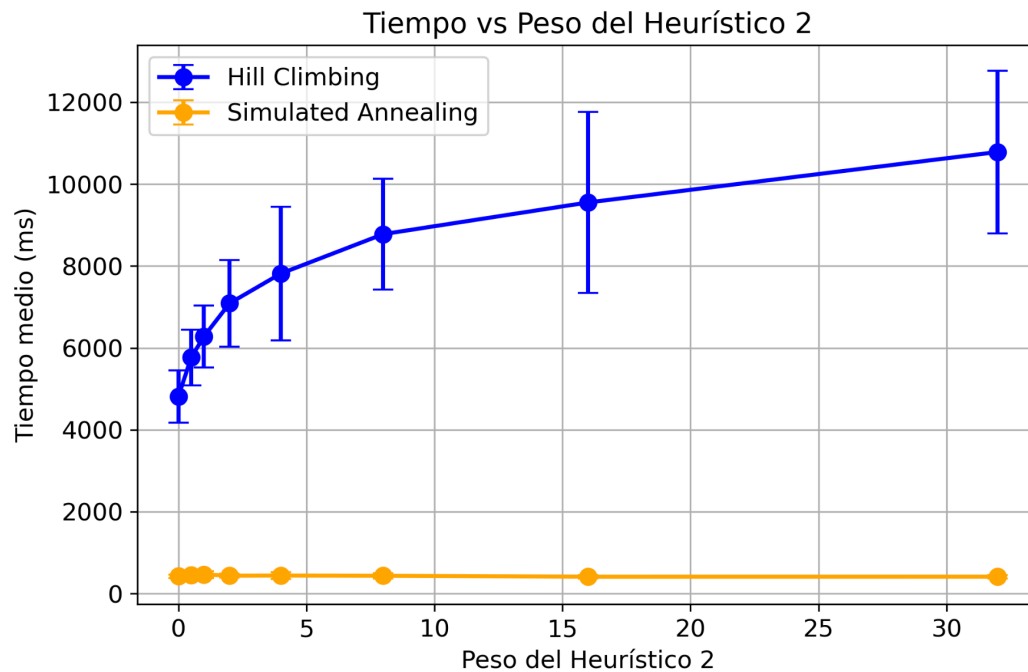
- **Escenario:** 100 grupos, 5 centros de rescate, 1 helicóptero por centro.
- **Algoritmos:** Comparativa entre Hill Climbing (HC) y Simulated Annealing (SA).
- **Heurística:** $H2 = H1 + w \times \max(\text{tiempo_llegada_P1})$
- **Variable independiente:** Pesos w pertenece al conjunto: $\{0, 0.5, 1, 2, 4, 8, 16, 32\}$.
- **Repeticiones:** 10 réplicas por cada peso y algoritmo para promediar el efecto de las semillas aleatorias.

c. Resultados:

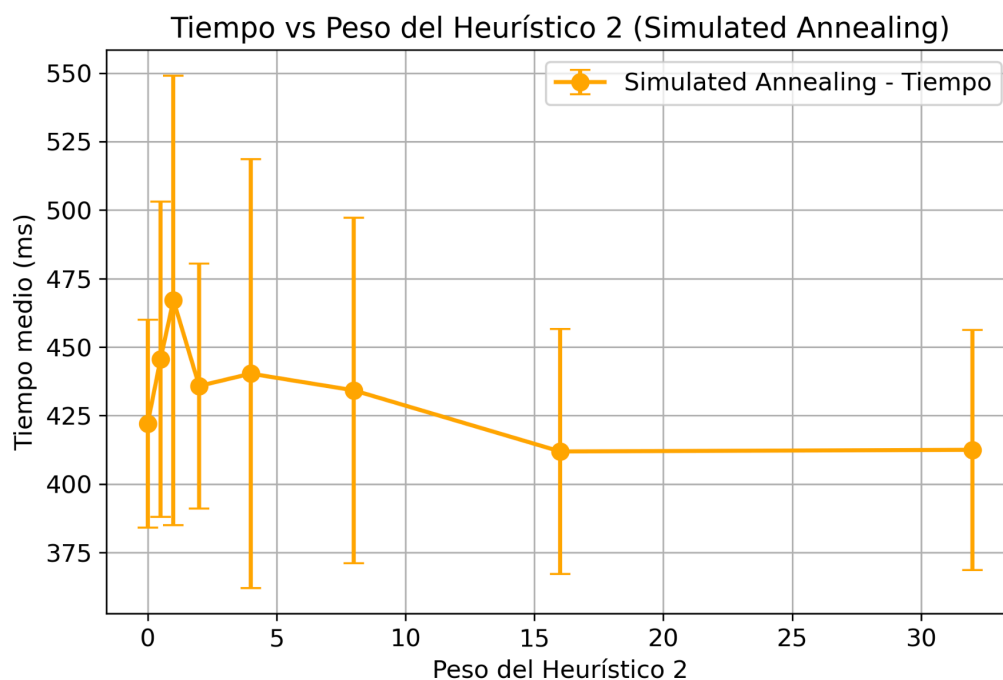
En el siguiente gráfico se muestra la relación del **coste medio de la solución** con el peso asignado al heurístico 2, para ejecuciones con Hill Climbing y ejecuciones con Simulated Annealing.



En el siguiente gráfico se puede observar la relación del **tiempo de ejecución del algoritmo** con el peso asignado al heurístico 2, para ejecuciones con Hill Climbing y ejecuciones con Simulated Annealing.



En el siguiente gráfico se puede observar la relación del **tiempo de ejecución del algoritmo** con el peso dado al heurístico 2, para ejecuciones con **Simulated Annealing**. Es una ampliación de la gráfica anterior para las ejecuciones con Simulated Annealing.



d. Conclusiones:

En relación con el coste (Heurística H2):

Se observa que el coste final crece de manera prácticamente **lineal** con el aumento del peso **w**. No hay diferencias significativas en la calidad de la solución final entre HC y SA en términos de la métrica **H2**, lo que indica que ambos algoritmos son capaces de encontrar soluciones que minimizan el término de prioridad 1 cuando se les incentiva con un peso alto.

En relación con el tiempo de ejecución:

- **Hill Climbing:** El tiempo de ejecución tiende a **aumentar** con el peso. Esto se debe a que la función de evaluación **H2** es más compleja de optimizar que **H1**; el algoritmo requiere evaluar más estados sucesores para encontrar mejoras en una superficie de búsqueda que ahora tiene en cuenta no solo la distancia total, sino el orden crítico de rescate.
- **Simulated Annealing:** Sorprendentemente, el tiempo de SA se mantiene **casi constante** (alrededor de 400-450 ms). Esto se explica porque SA tiene un número de iteraciones fijo (**steps = 10000**) definido por sus parámetros, por lo que su esfuerzo computacional es independiente de la complejidad de la función de coste, a diferencia de HC que sigue buscando hasta que no encuentra mejoras.

Conclusión Global del experimento:

El uso de **H2** es una herramienta efectiva para sesgar la solución hacia la urgencia médica. Mientras que HC se vuelve más lento al intentar resolver este compromiso, SA ofrece una alternativa con tiempos predecibles, aunque en este experimento específico ambos algoritmos han convergido a soluciones de calidad similar.

3. TRABAJO DE INNOVACIÓN

El trabajo de innovación se centrará en el proyecto ***Mapping the Mind of a Large Language Model*** desarrollado por *Anthropic*. Este proyecto aborda uno de los principales problemas de los modelos de lenguaje actuales: su carácter de “caja negra”.

En particular, el proyecto propone técnicas para identificar y aislar representaciones internas dentro del modelo, asociadas a conceptos concretos (como entidades, lenguajes de programación o ideas abstractas).

Para ello, emplea métodos de interpretabilidad como los autoencoders dispersos (*Sparse Autoencoders*), que permiten descomponer la actividad interna del modelo en características más comprensibles. Esta aproximación posibilita analizar cómo se representan los conceptos en la red neuronal e incluso influir en su comportamiento mediante la activación controlada de dichas características, lo que supone un avance relevante en términos de transparencia y seguridad en sistemas de inteligencia artificial.

En cuanto a la organización del trabajo, el grupo ha iniciado una fase de planificación y coordinación en la que se han distribuido las tareas entre los miembros. Paralelamente, se ha llevado a cabo una búsqueda inicial de información en fuentes primarias y especializadas, incluyendo noticias científicas y publicaciones de la propia empresa.